



**Hochschule
Hof**

University of
Applied Sciences

Software Engineering for Industrial Applications
Mandatory Internship SoSe-2026

Development of
AI-powered Retrieval-Augmented
Generation (RAG) system
“Software Validation Report”

Submitted To

Prof. Dr. Matthias Meitner
Hof University of Applied Science

Submitted By

Md Fahim Hossain
Student ID: 00154924

Table of Contents

1. Introduction & Company Overview	3
1.1. Introduction	3
1.2. Company Overview	3
1.3. Department Overview	4
1.4. Motivation for the Project	4
2. Role & Tasks	4
3. Project Overview	5
4. Tools & Frameworks	6
5. Code Review Process	8
5.1. Self-Review and Refactoring	8
5.2. Architectural Review and Code Quality Improvements	9
5.3. Final Review Before Feature Completion	10
6. Testing	11
6.1. Unit Testing	11
6.1.1. Testing the Document Chunking Service	12
6.1.2. Testing the PDF Extraction Service	12
6.1.3. Testing the Word Extraction Service	13
6.1.4. Testing the Excel Extraction Service	14
6.1.5. Testing the RAG Retrieval Service	15
6.2. API Testing	15
6.2.1. Testing the Embedding API Integration	16
6.2.2. Testing the Language Model API Integration	17
6.2.3. Testing API Failure Scenarios	18

List of Figures

Figure 1: Dispatching background document processing after a successful upload	9
Figure 2: Document processing with asynchronous embedding generation	10
Figure 3: Unit test verifying document chunk generation and metadata preservation	12

Figure 4: Unit test validating PDF text normalization.....	13
Figure 5: Unit test verifying Word document text extraction	13
Figure 6: Unit test verifying spreadsheet text extraction.....	14
Figure 7: Unit test verifying semantic document retrieval.	15
Figure 8: Mocking the embedding API response.	16
Figure 9: Verifying the embedding API request and response.....	17
Figure 10: Mocking the language model API response.	17
Figure 11: Verifying prompt construction before the language model request.	18
Figure 12: Verifying missing API credentials and unsupported provider configurations.....	18

1. Introduction & Company Overview

1.1. Introduction

As part of the Master's program in Software Engineering for Industrial Applications at Hof University of Applied Sciences, students are required to apply theoretical knowledge in a practical industrial environment through project-based work. This report documents the software specification and design activities performed during the development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document management and knowledge retrieval.

Modern organizations generate and maintain large volumes of technical documents, reports, manuals, process descriptions, and operational guidelines. Accessing relevant information from these documents often requires significant manual effort, leading to reduced efficiency and increased time spent searching for knowledge. To address this challenge, an intelligent document retrieval system was designed and developed that combines document processing, semantic search, and Artificial Intelligence technologies to provide context-aware answers based on organizational documents.

The project was developed during my work as a Working Student at Siemens Energy. The development process involved requirements analysis, software specification, system design, architecture planning, implementation, and validation of the proposed solution. This report focuses on the verification and validation activities performed during development. It describes the code review process, unit testing, API testing, performance testing, functional test cases, and validation activities carried out to ensure that the developed system satisfies its specified requirements and operates reliably under practical conditions.

The objective of the project was to develop a scalable and maintainable software solution that can process organizational documents, retrieve relevant information, and help users find answers through natural language queries.

1.2. Company Overview

Siemens Energy is one of the world's leading energy technology companies, operating across the entire energy value chain. The company develops innovative solutions for power generation, transmission, storage, and industrial applications while supporting the global transition toward sustainable and reliable energy systems.

With operations in more than 90 countries and a workforce of approximately 100,000 employees worldwide, Siemens Energy serves customers in both conventional and renewable energy sectors. The company provides technologies and services for gas turbines, steam turbines, power generation systems, grid infrastructure, hydrogen solutions, industrial applications, and digital transformation initiatives.

Digitalization plays a critical role in Siemens Energy's strategy. The company continuously invests in advanced software solutions, data-driven decision making, artificial intelligence, cloud technologies, and process automation to improve operational efficiency and support business innovation. These initiatives enable employees and stakeholders to access information more effectively, optimize workflows, and enhance organizational productivity.

1.3. Department Overview

The project was conducted within the SE GT DG OE department of Siemens Energy.

The department is involved in supporting digitalization initiatives, process optimization, and data-driven solutions that enhance operational efficiency across business functions. Various software tools, reporting systems, and analytical platforms are utilized to manage information, automate processes, and improve collaboration among stakeholders.

As part of ongoing digital transformation efforts, the department actively explores modern technologies such as cloud platforms, data analytics, artificial intelligence, and enterprise software systems. These technologies are used to streamline business processes, improve knowledge management, and support informed decision-making.

The development of the AI-powered document retrieval system aligns with these objectives by providing an intelligent mechanism for managing and accessing organizational knowledge. The project contributes to the broader goal of improving information accessibility and reducing the effort required to locate relevant content within large collections of enterprise documents.

1.4. Motivation for the Project

Organizations frequently store critical knowledge across numerous documents, including technical manuals, operational procedures, project documentation, and internal reports. Although these documents contain valuable information, retrieving specific content often requires manual searching, which can be time-consuming and inefficient.

The increasing adoption of Artificial Intelligence technologies has created opportunities to enhance document management systems through semantic understanding and context-aware information retrieval. Instead of relying solely on keyword-based searches, modern AI-assisted systems can understand user intent and identify relevant information based on contextual meaning.

The motivation behind this project was to design and develop a software solution capable of improving access to organizational knowledge while maintaining a user-friendly experience. By integrating document processing, semantic retrieval mechanisms, and AI-assisted response generation, the proposed system aims to support employees in obtaining accurate information quickly and efficiently.

The project also provided an opportunity to apply software engineering principles, including requirements engineering, system design, architectural modeling, and software development methodologies within a real industrial environment.

2. Role & Tasks

During my internship at Siemens Energy, I worked as a Working Student in the Digital Solutions team, contributing to both enterprise software development and data engineering activities. My responsibilities included supporting internal data analytics initiatives while leading the design and development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document

management and knowledge retrieval. Throughout the internship, I collaborated with software engineers and business stakeholders to analyse requirements, design software components, implement new features, and document the developed solution.

My primary responsibility was the development of the RAG system. I participated in the complete software development lifecycle, including requirements analysis, system design, backend development, user interface implementation, and technical documentation. The system was designed to provide secure and efficient access to enterprise knowledge through semantic search and AI-assisted question answering.

As part of this work, I developed the document management and processing workflow, enabling users to upload documents, monitor processing progress, and retrieve information through a conversational interface. I also implemented semantic retrieval, conversation management, and background processing features to support efficient document indexing and question answering. In addition, I contributed to improving the system architecture, optimizing database operations, and preparing the application for containerized deployment.

Alongside the RAG project, I supported enterprise data engineering activities involving Python, SQL, and Snowflake. My work included developing and validating data processing workflows, transforming business data for reporting purposes, and contributing to the development of Power BI dashboards in collaboration with domain experts.

Throughout the internship, I actively participated in team discussions, code reviews, and technical knowledge-sharing sessions. These activities improved both the software and my understanding of collaborative development practices. I also prepared technical documentation to support future maintenance and continued development of the application.

The combination of enterprise software development, AI-assisted document retrieval, and data engineering provided valuable experience in designing scalable software systems, collaborating within multidisciplinary teams, and applying software engineering principles to solve practical industrial challenges.

3. Project Overview

The project developed during this internship is an AI-powered Retrieval-Augmented Generation (RAG) system designed to improve enterprise document management and knowledge retrieval. The application enables users to upload organizational documents, process them automatically, and interact with their contents through a conversational interface. Instead of manually searching through lengthy reports or technical documentation, users can ask questions in natural language and receive responses generated from the relevant information contained within the uploaded documents.

The primary objective of the system is to improve information accessibility while ensuring that responses are generated from trusted organizational documents rather than relying solely on the knowledge of a language model. By combining semantic document retrieval with AI-assisted response generation, the

system provides context-aware, document-grounded answers while supplying only the most relevant document content for response generation.

Document Upload and Processing

Users can upload and manage enterprise documents through a centralized repository. After upload, the system automatically extracts document text, performs preprocessing, divides the content into semantic chunks, and prepares it for retrieval. This automated processing pipeline minimizes manual effort and ensures that documents become searchable once processing has been completed.

Semantic Retrieval and Question Answering

The system retrieves relevant document sections using semantic similarity rather than exact keyword matching, allowing users to ask questions in natural language. The retrieved document content is then provided to a language model, which generates context-aware responses grounded in the uploaded documents. The application also presents document references so that users can verify the origin of the retrieved information.

Conversation Management

The application provides a conversation-oriented interface where users can create multiple conversations, select one or more processed documents as knowledge sources, review previous discussions, and continue asking follow-up questions while preserving conversational context.

System Architecture

The system follows a modular architecture that separates document management, document processing, semantic retrieval, conversational interaction, and background processing into independent components. This design improves maintainability, scalability, and extensibility, allowing future enhancements to be incorporated with minimal impact on the existing system.

Overall, the developed RAG system combines automated document processing, semantic search, and AI-assisted question answering within a unified platform for enterprise knowledge retrieval. The solution improves information accessibility, reduces the effort required to locate relevant knowledge, and provides employees with an efficient and reliable way to interact with organizational documents.

4. Tools & Frameworks

The development and verification of the AI-powered document retrieval system relied on a combination of backend technologies, development tools, testing utilities, and supporting frameworks. These tools were selected to ensure that the application could be implemented efficiently while also providing mechanisms for debugging, validating functionality, and maintaining software quality throughout the development lifecycle.

Laravel Framework

Laravel 12 served as the primary backend framework for implementing the application. It provided the overall project structure, routing, authentication, middleware, database abstraction, queue management, and testing support. Its modular architecture enabled individual components to be

developed and verified independently while maintaining clean separation between business logic and presentation.

PostgreSQL and pgvector

PostgreSQL was used as the primary relational database for storing application data, including users, uploaded documents, conversations, and metadata. The pgvector extension enabled efficient storage and similarity search of document embeddings, allowing semantic document retrieval to be validated during functional testing. The database also supported verification of data integrity and consistency throughout the testing process.

AI Models

The system utilizes two different AI models during document processing and question answering. An embedding model generates vector representations of document chunks, while a language model produces context-aware responses using the retrieved information. During validation, these models were tested with different document collections to verify retrieval accuracy, citation generation, and response relevance.

Poppler and Tesseract OCR

Poppler was used to extract text from digitally generated PDF documents, while Tesseract OCR processed scanned documents containing image-based text. Both tools were validated using documents with different layouts and formats to ensure that extracted text was sufficiently accurate for subsequent indexing and semantic retrieval.

Queue Processing

Laravel's queue system was used to execute computationally intensive tasks such as document parsing, embedding generation, and indexing asynchronously. Background job processing allowed the application to remain responsive during large document uploads while simplifying verification of processing workflows and error handling.

Git and GitHub

Git was used throughout development to manage source code changes and maintain version history. GitHub served as the central repository for collaboration, feature integration, and code review. The version control workflow simplified tracking code modifications, reviewing changes before merging, and reverting implementations when necessary.

Docker

Docker was used to containerize the application and its supporting services, providing a consistent development and deployment environment. Containerization reduced environment-specific configuration issues and simplified validation by ensuring that the application behaved consistently across different development machines.

Development and API Testing Tools

Several development tools were used throughout implementation and verification:

- **Visual Studio Code** served as the primary integrated development environment for implementing and debugging the application.

- **Postman** was used to verify backend API endpoints, authentication mechanisms, request validation, and response handling.
- **PgAdmin** was used to inspect database records, verify data integrity, execute SQL queries, and validate vector storage and retrieval.
- **Browser Developer Tools** were used to debug frontend interactions, inspect network requests, monitor JavaScript execution, and validate user interface behavior.

Performance and Load Testing

Performance testing focused on evaluating the responsiveness of the application under concurrent user activity. Multiple document uploads, simultaneous chat requests, and background processing tasks were executed to observe application stability, queue execution, and database performance. These tests helped identify potential bottlenecks and verify that the system maintained acceptable response times during typical workloads.

5. Code Review Process

Software quality was continuously improved throughout the development of the AI-powered document retrieval system by performing regular code reviews during implementation. Rather than treating code review as a single activity before deployment, reviews were conducted whenever a new feature, enhancement, or bug fix was completed. This iterative review process helped maintain code readability, reduce defects, and ensure that newly developed components integrated correctly with the existing architecture.

Since the project followed an incremental development approach, every major functionality—including document upload, document processing, semantic retrieval, conversation management, OCR support, and administrative features—was reviewed before being considered complete. The review process primarily focused on maintainability, correctness, performance, and consistency with Laravel development practices.

The review activities were organized into three stages.

5.1. Self-Review and Refactoring

The first stage of the review process was performed immediately after implementing a new feature. Before considering the implementation complete, the code was manually reviewed to identify potential improvements and eliminate unnecessary complexity.

During this stage, the following aspects were verified:

- Compliance with Laravel coding conventions.
- Removal of duplicated logic.
- Clear naming of variables, methods, and classes.
- Proper separation of responsibilities between controllers, services, and background jobs.
- Exception handling for possible runtime failures.
- Database operations executed only when required.
- Readability and maintainability of the implementation.

A common design decision throughout the project was keeping controllers lightweight by limiting their responsibilities to request validation, authorization, and dispatching background processing tasks. Computationally expensive operations such as document parsing, text chunking, and embedding generation were intentionally moved into dedicated queue jobs. This approach simplified both development and later verification because each component had a clearly defined responsibility.

For example, after a document upload is validated and stored, the controller simply dispatches a background processing job instead of performing the complete document processing workflow synchronously.



```
ProcessDocumentJob::dispatch($document->id);

return redirect()
  ->route('documents.show', $document)
  ->with('success', 'Document uploaded successfully. Processing has started.');
```

```
return go(f, seed, [])
}
```

Figure 1: Dispatching background document processing after a successful upload

This implementation improves readability, reduces controller complexity, and prevents long-running operations from blocking the user interface.

5.2. Architectural Review and Code Quality Improvements

After the initial implementation, the project structure was reviewed from an architectural perspective to ensure that new functionality remained consistent with the overall system design.

The review concentrated on several important software engineering principles:

- Single Responsibility Principle for controllers, services, and jobs.
- Reusability of shared services.
- Consistent error handling.
- Proper database transaction management.
- Efficient interaction with external services.
- Clear separation between business logic and presentation logic.

During this stage, several improvements were introduced as the project evolved. For example:

- Business logic was gradually moved from controllers into dedicated service classes.
- Background processing was separated into queue jobs.
- Common functionality was reused instead of duplicated.
- Exception handling and logging were standardized.
- Database updates involving multiple operations were grouped into transactions.

The document processing workflow illustrates this design. After text extraction is completed successfully, the database updates are executed within a transaction before the next embedding generation job is dispatched.



```
DB::transaction(function ( ) use ( $document, $chunks ) {  
  
  // Store processed chunks  
  
  $document->forceFill([  
    'status' => Document::STATUS_CHUNKED,  
    'total_chunks' => count($chunks),  
  ]->save();  
  
});  
  
GenerateDocumentEmbeddingsJob::dispatch($document->id);
```

Figure 2: Document processing with asynchronous embedding generation

Using database transactions ensured that incomplete processing could not leave the system in an inconsistent state. Dispatching the embedding generation as a separate background job also improved scalability and simplified independent testing of each processing stage.

5.3. Final Review Before Feature Completion

Before a feature was considered complete, a final review was performed to verify both implementation quality and functional correctness. This review ensured that the feature integrated correctly with the remainder of the system and behaved as expected under realistic usage scenarios.

The final review included verification of:

- Correct execution of application workflows.
- Database consistency after user actions.
- Proper queue execution and background processing.
- Accurate handling of exceptional conditions.
- User interface behaviour and validation messages.
- Compliance with project coding standards.
- Compatibility with previously implemented features.

Particular attention was given to features involving multiple components, such as document upload, OCR processing, semantic indexing, and AI-assisted question answering. Since these features interact with several independent modules, successful completion required verification of the entire processing pipeline rather than individual functions alone.

Any issues identified during this stage were corrected before proceeding to the formal verification activities described in the next chapter. Once the code review process had been completed, automated unit tests, API integration tests, load testing, and functional test cases were executed to validate the correctness of the implemented features. Performing code reviews before testing reduced the number of defects entering the verification phase and improved the overall reliability and maintainability of the software.

6. Testing

Testing was carried out throughout the development of the AI-powered document retrieval system to verify the correctness, reliability, and robustness of the implemented software components. The testing process aimed to ensure that individual modules behaved as expected, external integrations functioned correctly, and the complete application satisfied its functional requirements before deployment.

To achieve this, multiple testing approaches were adopted throughout development. Automated unit tests were implemented using Laravel's PHPUnit testing framework to verify the behaviour of individual services in isolation. API testing validated communication with external AI services, while load testing and functional test cases evaluated the application's behaviour under realistic operating conditions.

Because the application consists of several independent components including document extraction, document chunking, semantic retrieval, vector embedding generation, and AI-assisted response generation testing was performed incrementally as each feature was completed. This modular approach simplified debugging, reduced regression issues, and improved the overall maintainability of the software.

The following sections describe the testing activities performed during the project.

6.1. Unit Testing

Unit testing focused on verifying the behaviour of individual software components independently before they were integrated into the complete document processing workflow. Laravel's built-in PHPUnit testing framework was used to automate these tests, allowing each service to be validated under controlled conditions without depending on other modules of the application.

Unlike integration testing, each unit test concentrated on a single responsibility of the corresponding service. This approach simplified debugging, enabled regression testing after code modifications, and ensured that individual components continued to function correctly throughout the development process.

The automated unit tests implemented during the project covered the core document processing services, including document chunking, text extraction from multiple document formats, OCR decision logic, and semantic retrieval. Together, these components form the foundation of the document processing pipeline and therefore required comprehensive verification before integration with the remaining system.

6.1.1. Testing the Document Chunking Service

The DocumentChunker service is responsible for dividing extracted document content into manageable semantic chunks while preserving the metadata required for later retrieval and citation generation. Since this service directly influences the quality of semantic search, automated tests were implemented to verify correct chunk generation for different document sources and extraction methods.

The following test verifies that page information and extraction metadata are correctly preserved during the chunking process.



```
$chunks = app(DocumentChunker::class)->chunkPages([
    ['page' => 5, 'content' => str_repeat('Page five sentence. ', 20)],
    ['page' => 6, 'content' => str_repeat('Page six sentence. ', 20)],
], chunkSize: 900, overlap: 100);

$this->assertNotEmpty($chunks);
$this->assertSame(5, $chunks[0]['page_start']);
$this->assertSame(6, $chunks[0]['page_end']);
$this->assertSame('pdf_text_extraction', $chunks[0]['metadata']['source']);
$this->assertSame(['native'], $chunks[0]['metadata']['extraction_methods']);
```

Figure 3: Unit test verifying document chunk generation and metadata preservation

Test Objectives

- Verify correct chunk generation.
- Confirm preservation of page boundaries.
- Validate extraction metadata.
- Verify document source identification.
- Ensure generated chunks remain suitable for semantic retrieval.

Expected Outcome

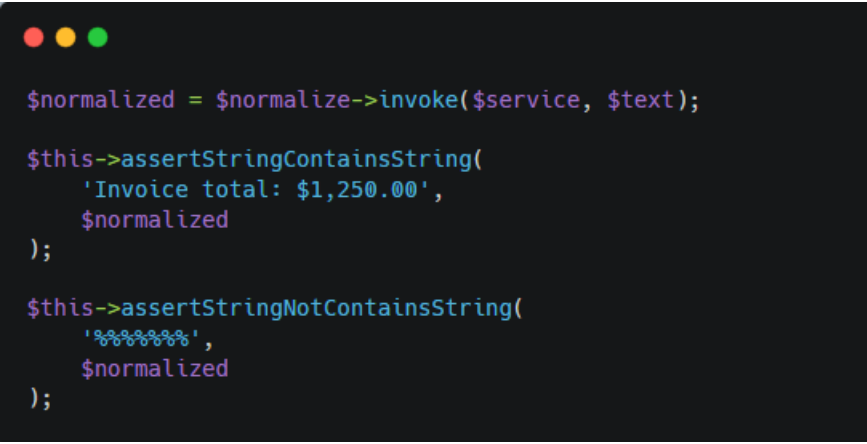
The generated chunks preserve document structure, page information, and extraction metadata, ensuring accurate semantic retrieval and citation generation.

The remaining unit tests for this service additionally verify OCR metadata preservation, spreadsheet metadata handling, and UTF-8 character integrity to ensure reliable processing of different document types.

6.1.2. Testing the PDF Extraction Service

The PdfExtractorService is responsible for extracting and normalizing textual content from PDF documents before further processing. Since extracted text may contain unwanted control characters, encoding artifacts, or formatting inconsistencies, automated tests were implemented to verify that the normalization process preserves meaningful content while removing extraction noise.

The following test validates the text normalization behaviour.



```

$normalized = $normalize->invoke($service, $text);

$this->assertStringContainsString(
    'Invoice total: $1,250.00',
    $normalized
);

$this->assertStringNotContainsString(
    '%%%%%%%%',
    $normalized
);

```

Figure 4: Unit test validating PDF text normalization

Test Objectives

- Verify removal of extraction artifacts.
- Preserve meaningful document content.
- Validate UTF-8 encoding.
- Remove invalid control characters.
- Improve extracted text quality.

Expected Outcome

The normalization process removes unwanted characters while preserving readable document content for subsequent indexing and semantic retrieval.

6.1.3. Testing the Word Extraction Service

The WordExtractorService extracts textual information from Microsoft Word documents while preserving the logical reading order of the original document. Automated tests were implemented to verify that document text was extracted correctly from supported Word formats and remained suitable for subsequent processing.

The unit tests validate successful extraction of textual content and confirm that the returned output is correctly formatted before entering the document chunking stage.



```

$pages = app(WordExtractorService::class)->extractPages($path);

$this->assertCount(1, $pages);
$this->assertNull($pages[0]['page']);
$this->assertSame('docx', $pages[0]['metadata']['source_type']);
$this->assertSame(
    'word_text_extraction',
    $pages[0]['metadata']['extraction_method']
);

$this->assertStringContainsString(
    'Mobile Purchase Notes',
    $pages[0]['content']
);

```

Figure 5: Unit test verifying Word document text extraction

Test Objectives

- Verify successful text extraction.
- Preserve document content.
- Validate extracted text format.
- Confirm compatibility with the document processing pipeline.

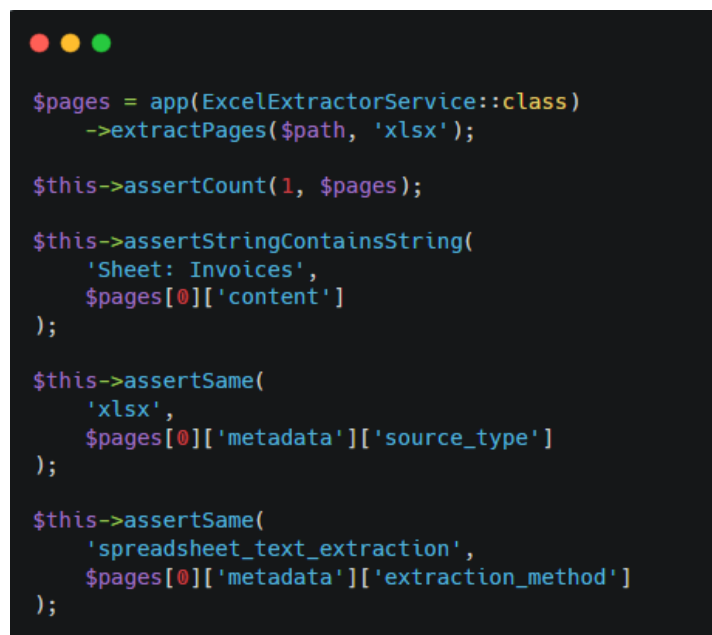
Expected Outcome

Text is extracted successfully from Word documents and prepared for semantic chunk generation without losing relevant information.

6.1.4. Testing the Excel Extraction Service

The ExcelExtractorService extracts structured data from spreadsheet documents while preserving worksheet information and row-level context. Automated tests were implemented to verify that spreadsheet data is converted into meaningful textual representations suitable for semantic indexing.

The unit tests confirm that worksheet names, row information, and extracted values are correctly preserved during processing.



```
$pages = app(ExcelExtractorService::class)
->extractPages($path, 'xlsx');

$this->assertCount(1, $pages);

$this->assertStringContainsString(
    'Sheet: Invoices',
    $pages[0]['content']
);

$this->assertSame(
    'xlsx',
    $pages[0]['metadata']['source_type']
);

$this->assertSame(
    'spreadsheet_text_extraction',
    $pages[0]['metadata']['extraction_method']
);
```

Figure 6: Unit test verifying spreadsheet text extraction

Test Objectives

- Verify spreadsheet text extraction.
- Preserve worksheet metadata.
- Validate row information.
- Confirm compatibility with semantic indexing.

Expected Outcome

Spreadsheet contents are transformed into structured textual representations while preserving contextual metadata required during retrieval.

6.1.5. Testing the RAG Retrieval Service

The RagRetrievalService is responsible for retrieving the most relevant document chunks based on semantic similarity before they are provided to the language model for response generation. As this component represents the core retrieval mechanism of the RAG system, automated tests were implemented to verify retrieval accuracy, document filtering, and result ranking.

The unit tests verify that only the appropriate document chunks are retrieved and that the returned results satisfy the specified similarity criteria.



```
config([
    'services.rag.top_k' => 6,
    'services.rag.summary_top_k' => 12,
]);

$service = new RagRetrievalService(
    Mockery::mock(EmbeddingService::class)
);

$this->assertSame(
    12,
    $service->limitForQuestion(
        'Summarize this document'
    )
);

$this->assertSame(
    6,
    $service->limitForQuestion(
        'What is the renewal deadline?'
    )
);
```

Figure 7: Unit test verifying semantic document retrieval.

Test Objectives

- Verify semantic retrieval accuracy.
- Confirm document filtering.
- Validate similarity ranking.
- Ensure correct retrieval of relevant document chunks.

Expected Outcome

The retrieval service consistently returns the most relevant document chunks for subsequent response generation while respecting the specified retrieval criteria.

6.2. API Testing

The AI-powered document retrieval system relies on two external AI services to perform semantic document retrieval and response generation. The first service generates vector embeddings for document chunks, while the second generates context-aware answers based on the retrieved document

content. Since both services communicate through external HTTP APIs, dedicated API testing was performed to verify successful communication, request construction, response processing, and exception handling.

Unlike the internal application components tested during unit testing, API testing focused on validating the interaction between the application and external AI services. Automated tests were implemented using Laravel's testing framework together with HTTP request mocking. This approach allowed the API integration to be verified without requiring actual network communication, resulting in faster, repeatable, and deterministic test execution.

The API tests validated successful responses, invalid configurations, request formatting, and error handling to ensure reliable integration with both external services.

6.2.1. Testing the Embedding API Integration

The embedding service is responsible for converting extracted document text into numerical vector representations that are later stored within the vector database for semantic similarity search. Automated tests were implemented to verify that the embedding service constructs the correct API request, sends the required parameters, and correctly processes the returned embedding vector.

The test simulates the external API response using Laravel's HTTP mocking functionality.

```
Http::fake([
    'generativelanguage.googleapis.com/*' => Http::response([
        'embedding' => [
            'values' => [0.1, -0.2, 0.3, 0.4, -0.5],
        ],
    ]),
]);
```

Figure 8: Mocking the embedding API response.

Test Objectives

- Verify correct API request generation.
- Validate successful embedding generation.
- Confirm correct parsing of the returned vector.
- Verify embedding dimensions.
- Ensure no real network communication is required during testing.

Expected Outcome

The embedding service successfully processes the simulated API response and returns the expected embedding vector for further processing.

After generating the embedding, the automated test verifies both the returned values and the correctness of the outgoing HTTP request.



```

$this->assertSame([0.1, -0.2, 0.3, 0.4, -0.5], $embedding);

Http::assertSent(function ($request) {
    return $request->hasHeader('x-goog-api-key', 'test-key')
        && $request['output_dimensionality'] === 5;
});

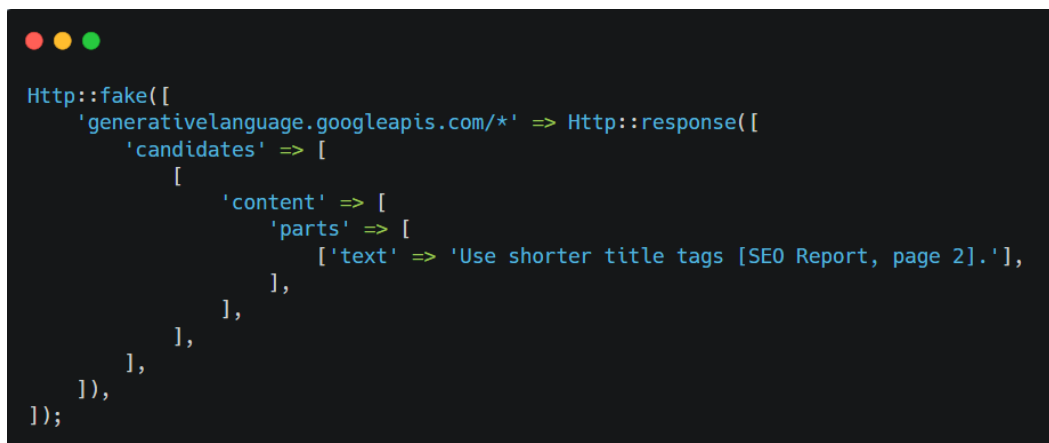
```

Figure 9: Verifying the embedding API request and response.

6.2.2. Testing the Language Model API Integration

After relevant document chunks have been retrieved, the language model generates the final response presented to the user. API testing verified that the application correctly constructs the prompt, communicates with the external language model, and processes the generated response.

The external service was simulated using HTTP mocking.



```

Http::fake([
    'generativelanguage.googleapis.com/*' => Http::response([
        'candidates' => [
            [
                'content' => [
                    'parts' => [
                        ['text' => 'Use shorter title tags [SEO Report, page 2].'],
                    ],
                ],
            ],
        ],
    ]),
]);

```

Figure 10: Mocking the language model API response.

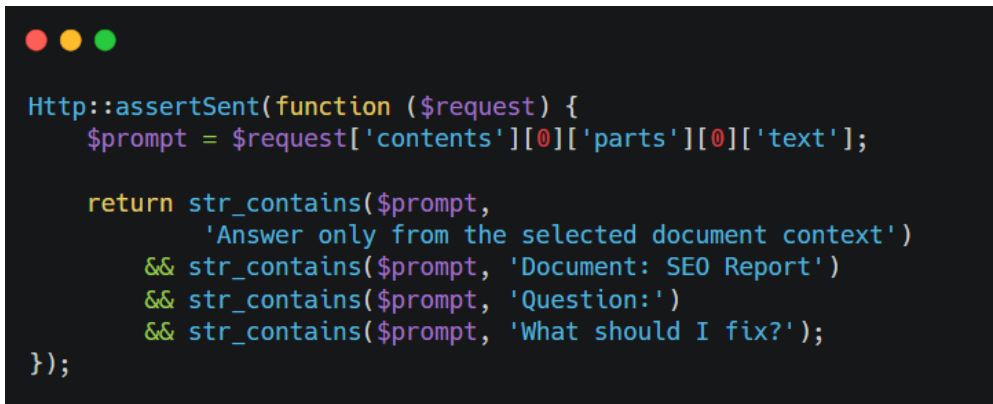
Test Objectives

- Verify successful API communication.
- Validate prompt construction.
- Confirm response parsing.
- Verify citation preservation.
- Ensure generated responses are returned correctly.

Expected Outcome

The language model returns a valid response based on the supplied document context, and the generated answer is successfully processed by the application.

The automated test also verifies that the generated prompt contains the required contextual information before being transmitted to the external service.



```
Http::assertSent(function ($request) {
    $prompt = $request['contents'][0]['parts'][0]['text'];

    return str_contains($prompt,
        'Answer only from the selected document context')
        && str_contains($prompt, 'Document: SEO Report')
        && str_contains($prompt, 'Question:')
        && str_contains($prompt, 'What should I fix?');
});
```

Figure 11: Verifying prompt construction before the language model request.

6.2.3. Testing API Failure Scenarios

In addition to successful communication, automated tests verified the application's behaviour when the external services were incorrectly configured or unavailable. Separate test cases confirmed that missing API credentials and unsupported providers resulted in controlled exceptions instead of unexpected application failures.

For example, the embedding service validates that an API key is configured before attempting communication with the external service.



```
$this->expectException(RuntimeException::class);
$this->expectExceptionMessage('API key is missing');

app(EmbeddingService::class)->embedText('hello');
```

Figure 12: Verifying missing API credentials and unsupported provider configurations

Similar validation tests were implemented for the language model service to verify missing API credentials and unsupported provider configurations.

Test Objectives

- Verify configuration validation.
- Confirm controlled exception handling.
- Prevent unnecessary external requests.
- Ensure predictable application behaviour.

Expected Outcome

Configuration errors are detected immediately, informative exceptions are generated, and the application fails safely without initiating invalid API requests.